

Project Spec: BMP390 Data Logger on the ESP32

Team	Payload Software
Hardware	Freenove ESP32 WROVER + Adafruit BMP390 breakout
Repo folder	prototypes/esp32_bmp390
Requirements	REQUIREMENTS.md in that folder (E32-1 to E32-13)
Time	about one day (roughly 5 hours), or spread over a few evenings

1. Purpose

Get hands on experience with the three things our flight computer does:

1. **Interface with hardware:** wire a sensor to a microcontroller over I2C.
2. **Read input:** use a driver library to get pressure and temperature.
3. **Save data on the board:** write every reading to a file in the ESP32's flash, then get it back onto a laptop.

You will learn each piece by following a short guide, then combine all three yourself in the skeleton program.

2. What you turn in

#	Deliverable
1	A photo of your paper schematic (Part 2)
2	Your finished <code>src/main.cpp</code> , building with no warnings
3	The <code>log.txt</code> from a 60 second recording
4	A plot of that file from <code>scripts/plot_data.py</code>
5	A two minute demo at the subteam meeting

3. Parts

Qty	Part	Notes
1	Freenove ESP32 WROVER	
1	Adafruit BMP390 breakout	Has its own regulator and I2C pullups
1	Breadboard	
4	Jumper wires	
1	USB cable	Must be a data cable, not charge only

4. The day plan

Part	What you do	Time	How
1	Set up PlatformIO, upload a test program	45 min	Follow Guide A
2	Draw the schematic on paper	15 min	Section 6
3	Wire the sensor, get readings on screen	45 min	Follow Guide B
4	Write and read a test file in flash	45 min	Follow Guide C
5	Combine everything in the skeleton	1.5 to 2 hrs	Your own work , Section 9
6	Plot the file, show the team	30 min	Section 10

Parts 1 to 4 are learning. Part 5 is where you prove you learned it: the guides do not show how to put the pieces together, and the skeleton only tells you what each piece must do.

5. Part 1: PlatformIO

Guide A: [Random Nerd Tutorials, Getting Started with VS Code and PlatformIO IDE for ESP32](#). Follow it through uploading the blink example. When it asks for a board, choose **Espressif ESP32 Dev Module**.

What PlatformIO is and why we use it

PlatformIO is a build system for microcontrollers that runs inside VS Code. It installs the compiler and upload tools for you, downloads the libraries a project needs, and builds, uploads and monitors with one click.

We use it instead of the Arduino IDE because:

- **Everyone builds the same thing.** The whole setup lives in one file, `platformio.ini`, committed to git, with the board and library versions pinned.
- **One tool for every board.** The flight computer is a Teensy 4.1. Switching is a new block in `platformio.ini`, not a new program.
- **Real project structure** in `src/` and `include/`, like the flight software.

The three buttons

On the blue bar at the bottom of VS Code:

Action	Icon	What it does
Build	✓	Compiles your code and the libraries into one firmware file for the ESP32. Nothing touches the board. Errors and warnings show up here.
Upload	→	Builds if needed, sends the firmware over USB into the ESP32's flash, and restarts the board. Your old program is replaced. Files already in LittleFS are not erased.
Monitor		Opens a two way text window to the board: you see what the program prints with <code>Serial</code> , and you can type back to it.

Common problems:

- **Port busy / access denied on upload:** the monitor is still open. Close it and upload again.
- **Garbage characters in the monitor:** the baud rate is not 115200.
- **"Failed to connect to ESP32":** hold the BOOT button while the upload says "Connecting...", release once it starts writing.
- **No COM port appears:** try another USB cable, or install the CH340 USB driver from Freenove's ESP32 tutorial.

Opening the project after a pull

PlatformIO only treats a folder as a project when `platformio.ini` is at the top of what VS Code has open. If you open the whole repo, the build, upload and monitor buttons either do not appear or do not know which project to use. So open the project folder itself:

1. Pull the repo as usual.
2. In VS Code: **File → Open Folder...**
3. Go into the repo, then `prototypes`, and select the `esp32_bmp390` folder (the one that contains `platformio.ini`). Click **Select Folder**.
4. VS Code reloads with that folder open. The first time, PlatformIO downloads the ESP32 tools and the Adafruit library, which takes a few minutes.
5. The ✓, → and  icons appear on the bottom bar.

Same result another way: click the PlatformIO ant icon in the left sidebar, open **PIO Home**, click **Open Project**, and pick the same folder.

Git still works. VS Code may ask whether to open the git repository it found in a parent folder. Click **Yes**, and pull, commit and push work as normal from the Source Control tab.

Want the whole repo open too? Use **File → Add Folder to Workspace...** and add `prototypes/esp32_bmp390`. PlatformIO finds the project and you choose it from the project switcher on the bottom bar. It works, but opening the project folder on its own is simpler the first time.

Our project file

When you open `prototypes/esp32_bmp390` (the folder with `platformio.ini`, not the repo root), this is what PlatformIO reads:

```
[env:esp32]
platform = espressif32      ; the chip family and its toolchain
board = esp32dev            ; generic ESP32, fits the Freenove WROVER
framework = arduino         ; lets us use Serial, Wire, millis()...
board_build.filesystem = littlefs ; the flash filesystem log.txt lives in
monitor_speed = 115200      ; must match Serial.begin()
build_flags = -Wall -Wextra ; show all warnings; we fix every one
lib_deps =
  adafruit/Adafruit BMP3XX Library @ ^2.1.6 ; the sensor library
```

6. Part 2: The schematic (on paper)

Before you plug anything in, draw the circuit by hand. It takes 15 minutes, and it is how we check a wiring plan before anything gets powered.

What to draw

- **One box for the ESP32.** Along its edge, write the pins you use: **3V3, GND, GPIO 13, GPIO 14.**
- **One box for the BMP390.** Along its edge, write **VIN, GND, SDA (SDI), SCL (SCK).**
- **Four lines** connecting them, each labeled with what it carries (power, ground, I2C data, I2C clock).
- In a corner: your name, the date, and the sensor's I2C address (0x77).

The layout looks like this. Draw the four connecting lines yourself:

ESP32 WROVER		BMP390 breakout
3V3	o	VIN
GND	o	GND
GPIO 13	o	SDA (SDI)
GPIO 14	o	SCL (SCK)

Answer these under your drawing

1. **Why 3V3 and not 5V?** (Hint: the breakout's I2C lines are pulled up to whatever voltage you give VIN, and the ESP32's pins only handle 3.3 V.)
2. **What do SDA and SCL each do?**
3. **Why are there no resistors in the drawing?** (Hint: I2C needs pullup resistors. Where are they?)

Have a teammate check your drawing, then wire the breadboard exactly as drawn. Take a photo of the page for deliverable 1.

7. Part 3: The sensor library

Guide B: [Random Nerd Tutorials, ESP32 with BMP388 Barometric Sensor](#). It uses the same Adafruit library, and the same code works for the BMP390. Adafruit's own [BMP388/BMP390 guide](#) is a good second reference.

Two changes from the guide, because of our wiring:

- The guide uses the ESP32's default I2C pins. We use 13 and 14, so start the bus with `Wire.begin(13, 14);` before starting the sensor.
- Start the sensor with `bmp.begin_I2C(0x77, &Wire)` so it uses that bus.

You are done with Part 3 when pressure and temperature print in the monitor. Expect about 101 kPa (the library prints Pa or hPa depending on the example; check the units) and room temperature. Breathe on the sensor and watch the temperature rise.

What the library contains

We use the official **Adafruit BMP3XX Library** ([GitHub](#)). It supports the BMP388 and the BMP390.

Piece	What it does
Adafruit_BMP3XX class	What you call: start the sensor, change settings, take a reading.
Bosch's bmp3 driver (bundled inside)	The chip maker's official code. Knows the chip's registers, reads its factory calibration, and turns raw numbers into real pressure and temperature.
Adafruit BusIO (installed automatically)	Does the actual I2C talking.

The calls you will use

Call	What it does
<code>bmp.begin_I2C(0x77, &Wire)</code>	Finds the sensor. Returns <code>false</code> if it does not answer: check wiring.
<code>bmp.performReading()</code>	Takes one fresh measurement of both values. Returns <code>false</code> if the read failed.
<code>bmp.pressure</code>	The last pressure, in pascals . Divide by 1000 for kPa.
<code>bmp.temperature</code>	The last temperature, in °C .

When you call `performReading()`, the library tells the chip to take one measurement, reads the raw bytes back over I2C, applies the calibration math, and stores the results in `pressure` and `temperature`. It takes a few milliseconds, which is plenty fast for 10 readings a second.

8. Part 4: Saving a file in flash

Guide C: [Random Nerd Tutorials, ESP32 Write Data to a File \(LittleFS\)](#). The guide uses the Arduino IDE, but the code runs unchanged in PlatformIO because our `platformio.ini` already selects LittleFS.

You are done with Part 4 when you can write a test file, reset the board, and read the same text back.

The ideas you need for Part 5:

Idea	Call
Mount the filesystem	<code>LittleFS.begin(true)</code>
Open a file, replacing the old one	<code>LittleFS.open("/log.txt", FILE_WRITE)</code>
Write a line	<code>logFile.println(...)</code> or <code>logFile.printf(...)</code>
Save to flash now	<code>logFile.flush()</code>
Finish with the file	<code>logFile.close()</code>

Why flush? Writes sit in memory until they are flushed or the file is closed. If the board resets first, those lines are lost. Flushing once a second limits the loss to about one second.

9. Part 5: Your program

Now combine the three pieces. Everything is in one file, `src/main.cpp`. The parts marked PROVIDED are done; each `TODO(E32-xx)` is yours, and the tag names the requirement it satisfies.

What your program does: read pressure and temperature 10 times a second for 60 seconds, write every reading to `/log.txt`, then close the file. Typing `d` afterward prints the file.

The file format is the one the flight logger uses, so our plotting script reads it without changes:

```
time_us,sensor,value,status
0,PRESSURE,101.3251,OK
0,TEMPERATURE,23.41882,OK
100012,PRESSURE,101.3248,OK
100012,TEMPERATURE,23.42011,OK
```

Why `micros()` and not `delay()`? `delay(100)` waits 100 ms *plus* however long the reading and writing took, so the rate drifts below 10 Hz. Scheduling each reading at a fixed time keeps it at 10 Hz no matter how long each one takes.

The skeleton

This is `src/main.cpp` as it sits in the repo. Edit the file in the repo, not this document.

```
// ESP32 + BMP390 logger
//
// Reads pressure and temperature 10 times a second for 60 seconds and saves
// every reading to /log.txt in the ESP32's flash. Type d in the serial
// monitor afterward to print the file.
//
// Requirements: REQUIREMENTS.md. Each TODO names the requirement it meets.
// Guides that teach each piece: PROJECT_SPEC.md, Part 3 and Part 4.

#include <Arduino.h>
#include <Wire.h>
#include <LittleFS.h>
#include <Adafruit_BMP3XX.h>
```

```

// ----- settings
// One named constant per setting. No magic numbers further down.

const int      SDA_PIN      = 13;      // E32-2
const int      SCL_PIN      = 14;      // E32-2
const uint8_t  BMP_ADDRESS = 0x77;     // E32-2 (0x76 if SDO is tied to GND)

const uint32_t SAMPLE_PERIOD_US = 100000; // 10 Hz (E32-5)
const uint32_t RUN_TIME_MS      = 60000;  // stop after 60 s (E32-10)
const uint32_t FLUSH_INTERVAL_MS = 1000;   // save to flash every second (E32-11)

const char*    LOG_PATH = "/log.txt";      // E32-7

// ----- state

Adafruit_BMP3XX bmp;
File logFile;

bool      recording      = false;
uint32_t  startUs        = 0;  // micros() when recording started
uint32_t  startMs        = 0;  // millis() when recording started
uint32_t  nextSampleUs   = 0;  // when the next reading is due
uint32_t  lastFlushMs    = 0;  // when the file was last saved to flash

// ----- provided helpers

// Write one line of the log: time_us,sensor,value,status (E32-8).
// %.7g keeps 7 significant digits, enough for 0.1 Pa on ~101 kPa.
void writeline(uint32_t timeUs, const char* sensor, double value, const char* status) {
    logFile.printf("%lu,%s,%.7g,%s\n", (unsigned long)timeUs, sensor, value, status);
}

// Save and close the file. The only way recording ends (E32-10).
void stopRecording() {
    logFile.flush();
    logFile.close();
    recording = false;
    Serial.println("Recording stopped. Type d to print the file.");
}

// Print the whole log between two marker lines, so it can be copied into a
// .txt on your laptop (E32-12).
void dumpLog() {
    File f = LittleFS.open(LOG_PATH, FILE_READ);
    if (!f) {
        Serial.println("No log file yet.");
        return;
    }
    Serial.printf("BEGIN FILE %s\n", LOG_PATH);
    while (f.available()) {
        Serial.write(f.read());
    }
    Serial.println("END FILE");
    f.close();
}

// ----- setup

void setup() {
    Serial.begin(115200);
    delay(500); // give the serial monitor a moment to attach

```

```

Serial.println("ESP32 BMP390 logger");

// TODO(E32-2): Start I2C on our pins.
// Wire.begin(SDA_PIN, SCL_PIN);

// TODO(E32-3): Start the sensor with bmp.begin_I2C(BMP_ADDRESS, &Wire).
// If it returns false, print "BMP390 not found, check wiring", wait one
// second, and try again. Keep trying until it works.
// (A while loop around begin_I2C does this.)

// PROVIDED: sensor settings. More oversampling = less noise but slower.
bmp.setTemperatureOversampling(BMP3_OVERSAMPLING_2X);
bmp.setPressureOversampling(BMP3_OVERSAMPLING_4X);
bmp.setIIRFilterCoeff(BMP3_IIR_FILTER_DISABLE);

// TODO(E32-4): Mount the filesystem with LittleFS.begin(true).
// If it returns false, print "LittleFS mount failed" and return.

// TODO(E32-7): Open the log for writing, which replaces any old file:
// logFile = LittleFS.open(LOG_PATH, FILE_WRITE);
// If (!logFile), print "Could not open log file" and return.

// TODO(E32-8): Write the header line to the file:
// time_us,sensor,value,status

// PROVIDED: start the clocks.
startUs = micros();
startMs = millis();
nextSampleUs = startUs;
lastFlushMs = startMs;
recording = true;
Serial.println("Recording to /log.txt for 60 s...");
}

// ----- loop

void loop() {
// PROVIDED: d prints the file, once recording is over.
if (Serial.available() && Serial.read() == 'd') {
    if (recording) {
        Serial.println("Still recording. Wait for it to finish.");
    } else {
        dumpLog();
    }
}

if (!recording) {
    return;
}

// TODO(E32-10): If millis() - startMs >= RUN_TIME_MS, call
// stopRecording() and return.

// TODO(E32-5): Only read when it is time. If micros() has not reached
// nextSampleUs yet, return. Otherwise move nextSampleUs forward by
// SAMPLE_PERIOD_US. Do not use delay() here.
//
// if ((int32_t)(micros() - nextSampleUs) < 0) return;
// nextSampleUs += SAMPLE_PERIOD_US;
//
// (Why the subtraction? micros() wraps back to 0 every ~71 minutes;
// subtracting still gives the right answer when it does.)

```

```

const uint32_t t = micros() - startUs; // time since recording started

// TODO(E32-6, E32-9): Take a reading with bmp.performReading().
// * If it returns true, write two lines with writeLine():
//     PRESSURE    bmp.pressure / 1000.0    (library gives Pa, we want kPa)
//     TEMPERATURE bmp.temperature         (already in C)
// both with status "OK".
// * If it returns false, still write both lines, with value 0 and
// status "COMM_ERROR". Never skip a line just because the read failed.
(void)t; // delete this line once you use t

// TODO(E32-11): If millis() - lastFlushMs >= FLUSH_INTERVAL_MS, call
// logFile.flush() and set lastFlushMs = millis(). Flushing makes sure a
// reset or unplugged cable loses at most about one second of data.
}

```

10. Part 6: Check and demo

After the 60 s run, type `d` in the monitor. Copy everything between `BEGIN FILE` and `END FILE` into a file called `log.txt` on your laptop, then from the repo root run:

```
python scripts/plot_data.py log.txt
```

The script prints each stream's measured rate and saves a plot.

Done when:

- ☐ Paper schematic checked by a teammate and photographed.
- ☐ Builds with no warnings.
- ☐ The file has the header line and about 1,200 lines (600 readings × 2).
- ☐ Pressure is about 101 kPa and temperature is near room temperature.
- ☐ Breathing on the sensor during recording shows up in the plot.
- ☐ `plot_data.py` reports about 10 Hz for both streams.
- ☐ Pulling the SDA wire during a recording gives `COMM_ERROR` lines, and the recording still finishes and closes the file.

11. Troubleshooting

Symptom	Likely cause
No build, upload or monitor icons	You opened the repo root. Open <code>prototypes/esp32_bmp390</code> instead (Section 5).
"BMP390 not found" forever	SDA and SCL swapped, a loose wire, or wrong address (try 0x76).
Upload fails, port busy	Close the monitor first.
Monitor shows garbage	Baud rate is not 115200.
Rate well under 10 Hz	A <code>delay()</code> in <code>loop()</code> , or printing every reading to Serial.
File is empty or cut short	The file was never flushed or closed.

12. Finished early?

- `prototypes/esp32_bmp390_full` is the same project split into modules, with the stretch requirements (numbered files, BOOT button stop, fault counting, file commands) as TODOs.
- `prototypes/esp32_bmp390_register_driver` drops the library entirely: you write the driver from the BMP390 datasheet, with tests that run on your laptop.